Git & GitHub Course Index (15 Modules)

Module 1: Introduction to Version Control, Git & GitHub

- What is Version Control?
- Types of Version Control Systems
- What is Git?
- History of Git
- Features of Git
- Advantages of Git
- What is GitHub?
- Git vs GitHub
- Git Architecture
- Real-world Use Cases

Module 2: Installing Git & Initial Configuration

- Installing Git (Windows/Linux/macOS)
- Git Bash
- Configure Username & Email
- Verify Installation
- First Git Repository
- Git Configuration Commands

Module 3: Git Repository Fundamentals

- Local Repository
- Remote Repository
- Working Directory
- Staging Area
- Git Objects
- Repository Structure

Module 4: Basic Git Commands

- `git init`
- `git status`

- `git add`
- `git commit`
- `git log`
- `git diff`
- `git help`

Module 5: Branching and Merging

- Branches
- Create Branch
- Switch Branch
- Merge Branch
- Delete Branch
- Merge Conflicts

Module 6: GitHub Basics

- Create GitHub Account
- Create Repository
- Push Local Project
- Clone Repository
- Fork Repository
- Pull Requests

Module 7: Remote Repository Management

- `git remote`
- `git push`
- `git pull`
- `git fetch`
- `git clone`
- Upstream Repository

Module 8: Git Collaboration

- Team Workflow

- Pull Requests
- Code Review
- Merge Requests
- Collaboration Best Practices

Module 9: Advanced Git Commands

- `git stash`
- `git reset`
- `git revert`
- `git cherry-pick`
- `git rebase`
- `git tag`

Module 10: Git Internals

- Commit Hash
- SHA-1
- Blob
- Tree
- Commit Object
- HEAD
- Index

Module 11: GitHub Features

- Issues
- Projects
- Wiki
- Discussions
- Releases
- GitHub Pages
- GitHub Actions (Introduction)

Module 12: Git Best Practices

- Commit Message Standards

- Branch Naming
- `.gitignore`
- Repository Structure
- Security Tips

Module 13: Real-World Git Workflow

- Feature Branch Workflow
- Git Flow
- GitHub Flow
- Open Source Contribution
- Team Collaboration

Module 14: Git & GitHub Projects

- Upload React Project
- Upload Python Project
- Portfolio Repository
- Open Source Contribution
- Team Project

Module 15: Git & GitHub Interview Preparation

- Top Interview Questions
- Practical Scenarios
- Common Mistakes
- Cheat Sheet
- Career Guidance

Module 1: Introduction to Version Control, Git & GitHub

Learning Objectives

After completing this module, you will be able to:

- Understand the concept of Version Control.

- Learn why Version Control is important in software development.
- Differentiate between Local, Centralized, and Distributed Version Control Systems.
- Understand Git and its features.
- Learn the history of Git.
- Understand GitHub and its purpose.
- Differentiate between Git and GitHub.
- Understand Git Architecture.
- Explore real-world use cases of Git and GitHub.

1.1 Introduction

Modern software development involves multiple developers working on the same project simultaneously. Without a version control system, managing source code becomes difficult because changes can overwrite each other, causing conflicts and data loss.

Version Control Systems (VCS) solve this problem by tracking changes made to files, allowing developers to collaborate efficiently, maintain a complete history of modifications, and restore previous versions whenever required.

Today, almost every software company—from startups to global enterprises—uses Git as its version control system.

Definition

Version Control is a system that records changes made to files over time, allowing users to track modifications, compare versions, and restore previous versions if necessary.

Real-Time Example

Suppose a team of five developers is building an E-commerce website.

- Developer A works on Login.
- Developer B develops the Payment Module.
- Developer C designs the Home Page.
- Developer D develops Product Search.
- Developer E fixes bugs.

Without Version Control:

- Files may overwrite each other.

- Code conflicts increase.
- Previous versions may be lost.

With Git:

- Every developer works independently.
- Changes are merged safely.
- Every modification is tracked.
- Older versions can be restored.

1.2 Why Version Control?

Version Control provides many advantages.

Benefits

- Tracks every code change.
- Prevents accidental data loss.
- Supports teamwork.
- Maintains project history.
- Simplifies debugging.
- Enables easy rollback to previous versions.
- Improves software quality.
- Supports parallel development through branches.

1.3 Types of Version Control Systems

There are three major types of Version Control Systems.

1. Local Version Control System (LVCS)

In Local Version Control, changes are stored only on a single computer.

Architecture

Developer

Local Repository

|

▼

Version History

Advantages

- Simple to use.
- No internet required.

Disadvantages

- No collaboration.
- Risk of data loss if the computer fails.

---

2. Centralized Version Control System (CVCS)

A central server stores all project versions.

Examples:

- SVN (Subversion)
- CVS

Architecture

Developer A

|

Developer B

|

Developer C

\

▼

Central Server

Advantages

- Easy collaboration.
- Centralized management.

Disadvantages

- Single point of failure.
- Server downtime affects all developers.

---

3. Distributed Version Control System (DVCS)

Every developer has a complete copy of the repository.

Example:

- Git

Architecture

Developer A

|

Developer B

|

Developer C

|

▼

Git Repository

(Complete Copy for Everyone)

Advantages

- No single point of failure.
- Faster operations.
- Offline work supported.
- Better collaboration.

1.4 What is Git?

Git is a **Distributed Version Control System (DVCS)** developed to manage source code efficiently.

It enables developers to:

- Track changes.
- Work offline.
- Create branches.
- Merge code.
- Restore previous versions.
- Collaborate with teams.

Definition

Git is a distributed version control system used to track changes in source code during software development.

1.5 History of Git

Git was created by **Linus Torvalds** in **2005**.

Reason:

The Linux Kernel project required a fast, reliable, distributed version control system.

Git quickly became the world's most popular version control system.

1.6 Features of Git

Git provides many powerful features.

Key Features

- Distributed architecture.
- High performance.
- Branching and merging.

- Fast commits.
- Complete project history.
- Data integrity.
- Offline support.
- Lightweight.
- Secure.

1.7 Advantages of Git

- Free and Open Source.
- Platform Independent.
- Supports Collaboration.
- Fast Performance.
- Easy Rollback.
- Branch-based Development.
- Strong Community Support.
- Widely Used in Industry.

1.8 What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories.

It allows developers to:

- Store code online.
- Collaborate with teams.
- Review code.
- Track issues.
- Manage projects.
- Contribute to open-source software.

GitHub uses Git as its version control engine.

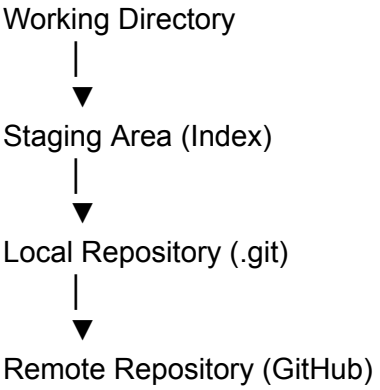
1.9 Git vs GitHub

Git	GitHub
Version Control System	Cloud Hosting Platform

Works locally	Works online
Tracks code changes	Stores Git repositories
Can be used without internet	Internet required for cloud features
Command-line tool	Web-based platform

1.10 Git Architecture

Git consists of three main areas.



Components

Working Directory

Where developers create and modify project files.

Staging Area

Temporary area where selected changes are prepared before committing.

Local Repository

Stores the complete version history on the local computer.

Remote Repository

Stores the project online (e.g., GitHub) for collaboration and backup.

1.11 Git Workflow

Create File

↓

Modify File

↓

Stage Changes

↓

Commit Changes

↓

Push to GitHub

↓

Collaborate

1.12 Applications of Git & GitHub

Git and GitHub are widely used in:

- Software Development
- DevOps
- Cloud Computing
- Data Science
- Machine Learning
- Mobile App Development
- Web Development
- Open Source Projects

1.13 Best Practices

- Commit changes frequently.
- Write meaningful commit messages.
- Use branches for new features.
- Pull the latest changes before starting work.
- Push code regularly to GitHub.
- Never commit sensitive information such as passwords or API keys.
- Use `.gitignore` to exclude unnecessary files.

1.14 Common Mistakes

❌ Editing code directly on the main branch.

❌ Forgetting to commit changes.

❌ Using unclear commit messages like "update" or "fix".

❌ Committing sensitive credentials.

❌ Ignoring merge conflicts.

Real-Time Scenario

A software company has a team developing an Online Banking Application.

Workflow:

1. Developers clone the repository from GitHub.
2. Each developer creates a separate branch.
3. New features are developed independently.
4. Changes are committed locally.
5. Code is pushed to GitHub.
6. Pull Requests are reviewed.
7. Approved changes are merged into the main branch.

This workflow ensures safe collaboration and maintains a reliable project history.

Interview Questions

1. What is Version Control?

Answer:

Version Control is a system that records changes to files over time, allowing users to track, manage, and restore previous versions.

---

2. What is Git?

Answer:

Git is a distributed version control system used to track changes in source code and support collaborative software development.

---

3. What is GitHub?

Answer:

GitHub is a cloud-based platform that hosts Git repositories, enabling collaboration, code sharing, version control, and project management.

---

4. What is the difference between Git and GitHub?

Answer:

Git is a version control tool that works locally, while GitHub is an online platform that hosts Git repositories and provides collaboration features.

---

5. Why is Git preferred over traditional version control systems?

Answer:

Git offers distributed development, faster performance, offline access, strong branching and merging capabilities, and better collaboration, making it the preferred choice for modern software development.

Practical Lab

Task 1

Research three popular Version Control Systems (Git, SVN, Mercurial) and compare their features.

Task 2

Draw the architecture of Git.

Task 3

List five advantages of using Git in software development.

Task 4

Write five differences between Git and GitHub.

Task 5

Describe a real-world software development workflow using Git and GitHub.

Module 2: Installing Git and Initial Configuration

Learning Objectives

After completing this module, you will be able to:

- Understand Git installation requirements.
- Install Git on Windows, Linux, and macOS.
- Verify Git installation.
- Configure Git with your username and email.
- Understand Git configuration levels.
- Initialize a Git repository.

- Create your first Git project.
- Verify Git configuration.

2.1 Introduction

Before using Git, it must be installed on your system.

Git is available for:

- Windows
- Linux
- macOS

After installation, Git needs some basic configuration, such as your **username** and **email address**, because Git records this information with every commit.

Real-Time Example

A software developer joins a company.

Before writing code, they:

- Install Git.
- Configure their name and email.
- Create a local repository.
- Connect it to GitHub.

Only then do they start developing the project.

2.2 System Requirements

Git has minimal hardware requirements.

Component	Minimum Requirement
Operating System	Windows 10/11, Linux, macOS
RAM	2 GB

Storage	500 MB Free Space
Internet	Required for GitHub

2.3 Installing Git on Windows

Step 1

Download Git from the official website:

https://git-scm.com

Step 2

Run the installer.

Choose:

- Next
- Install
- Finish

Most default options are suitable for beginners.

Step 3

Open:

Git Bash

or

Command Prompt

2.4 Installing Git on Ubuntu/Debian

Update package list:

sudo apt update

Install Git:

sudo apt install git

2.5 Installing Git on Fedora

sudo dnf install git

2.6 Installing Git on macOS

Using Homebrew:

brew install git

2.7 Verify Git Installation

After installation, verify Git.

Command:

git --version

Example Output:

git version 2.50.1

This confirms Git is installed successfully.

2.8 Git Configuration

Git stores user information for every commit.

Configure Username:

git config --global user.name "Prasanna"

Configure Email:

git config --global user.email "prasanna@example.com"

Replace the example values with your own name and email address.

2.9 Git Configuration Levels

Git supports three configuration levels.

Level	Scope
System	Entire Computer
Global	Current User
Local	Current Repository

System Configuration

git config --system

Administrator privileges are usually required.

Global Configuration

git config --global user.name "Prasanna"

Applies to all repositories for the current user.

Local Configuration

git config --local user.name "Developer"

Applies only to the current repository.

2.10 View Git Configuration

Display all configuration settings:

git config --list

Display Username:

git config user.name

Display Email:

git config user.email

2.11 Initialize a Git Repository

Create a project folder:

mkdir MyProject

Go inside it:

cd MyProject

Initialize Git:

git init

Output:

Initialized empty Git repository

Git creates a hidden folder:

.git

This folder stores the repository's history and configuration.

2.12 First Git Project

Create a file:

touch README.md

Check repository status:

git status

Example Output:

Untracked files:

README.md

Git now knows a new file exists but is not yet being tracked.

2.13 Repository Initialization Workflow

Install Git

Configure User

Create Folder

git init

Repository Ready

2.14 Common Git Commands (So Far)

Command	Purpose
`git --version`	Verify installation
`git config --global user.name`	Set username
`git config --global user.email`	Set email
`git config --list`	Display configuration
`mkdir`	Create folder
`cd`	Change directory
`git init`	Initialize repository
`git status`	Display repository status

2.15 Best Practices

- Install the latest stable version of Git.
- Configure your correct username and email.
- Verify the installation before creating repositories.
- Use meaningful project folder names.
- Keep one Git repository for one project.
- Do not manually edit the `.git` directory.

2.16 Common Mistakes

❌ Forgetting to configure username and email.

❌ Running `git init` in the wrong directory.

❌ Deleting the `.git` folder accidentally.

❌ Using different email addresses across projects without intention.

❌ Forgetting to verify the Git installation.

Real-Time Scenario

A new developer joins a software company.

They perform the following steps:

Install Git:

git --version

Configure identity:

git config --global user.name "John"
git config --global user.email "john@example.com"

Create project:

mkdir EmployeePortal
cd EmployeePortal
git init

The project is now ready for version control.

Interview Questions

1. Why is Git configuration required?

Answer:

Git uses the configured username and email to identify the author of each commit, making it easier to track changes and collaborate with other developers.

---

2. What is the purpose of `git init`?

Answer:

The `git init` command creates a new Git repository by initializing a hidden `.git` directory that stores version history and repository metadata.

---

3. What is stored inside the `.git` directory?

Answer:

The `.git` directory contains commit history, branch information, configuration files, references, and all metadata required for version control.

---

4. What is the difference between Global and Local Git configuration?

Answer:

- **Global configuration** applies to all repositories for the current user.
- **Local configuration** applies only to a specific repository and overrides global settings when configured.

---

5. How do you verify that Git is installed correctly?

Answer:

Run:

git --version

If Git is installed correctly, it displays the installed version.

Practical Lab

Task 1

Install Git on your operating system.

Task 2

Verify the installation using:

git --version

Task 3

Configure your Git username and email.

Task 4

Create a folder named `GitPractice` and initialize it as a Git repository.

Task 5

Use `git config --list` to verify your configuration.

Module 3: Git Repository Fundamentals

Learning Objectives

After completing this module, you will be able to:

- Understand what a Git repository is.
- Learn the structure of a Git repository.
- Differentiate between Local and Remote repositories.
- Understand the Working Directory, Staging Area, and Local Repository.
- Learn Git Objects (Blob, Tree, Commit, Tag).
- Understand the HEAD pointer.
- Learn the Git lifecycle.
- Understand how Git stores project history.

3.1 Introduction

A **Git Repository** is the storage area where Git keeps your project files, commit history, branches, and configuration.

Whenever you initialize Git using:

git init

Git creates a hidden folder called:

.git

This folder contains all the information required to manage the project's version history.

Definition

A **Git Repository** is a database that stores all versions of a project, including files, commits, branches, tags, and metadata.

Real-Time Example

A software company develops an **Online Shopping Application**.

The project contains:

- Login Module
- Product Module
- Payment Module
- Admin Panel

Git stores every modification made to these modules, allowing developers to return to previous versions if necessary.

3.2 Types of Git Repositories

Git mainly uses two repositories.

1. Local Repository

Stored on the developer's computer.

Characteristics:

- Works offline.
- Stores complete project history.
- Allows commits without internet.

Example:

C:\Projects\OnlineStore

---

2. Remote Repository

Stored on cloud platforms.

Examples:

- GitHub
- GitLab
- Bitbucket

Characteristics:

- Shared among team members.
- Used for collaboration.
- Requires internet connectivity.

3.3 Local vs Remote Repository

Local Repository	Remote Repository
Stored on local computer	Stored on GitHub
Offline access	Online access
Private development	Team collaboration
Fast operations	Shared repository

3.4 Git Repository Structure

```
MyProject/
│
├── .git/
├── src/
├── README.md
├── package.json
└── index.html
```

Important Components

- **Project Files** → Source code.
- **.git Folder** → Git database.
- **Configuration Files** → Git settings.
- **Commit History** → Previous versions.

3.5 The `.git` Directory

The `.git` directory is automatically created after running:

git init

It contains:

.git/

├── objects/

├── refs/

├── hooks/

├── config

├── HEAD

└── index

Purpose

- Stores commits.
- Stores branches.
- Stores configuration.
- Stores staging information.
- Stores repository metadata.

⚠️ **Never delete or manually modify the `.git` folder unless you understand its purpose.**

3.6 Git Three Working Areas

Git manages files using three important areas.

Working Directory

Staging Area

Local Repository

Working Directory

The place where developers create and edit project files.

Example:

index.html

style.css

app.js

Files here may be:

- New
- Modified
- Deleted

Staging Area (Index)

The staging area is a temporary storage location where selected changes are prepared before committing.

Command:

git add filename

Purpose:

- Select specific changes.
- Prepare for commit.
- Review changes before saving permanently.

Local Repository

After committing:

git commit

Git permanently stores the snapshot inside the local repository.

3.7 Git Workflow

Create File

Modify File

Working Directory

git add

Staging Area

git commit

Local Repository

git push

Remote Repository

3.8 Git Objects

Git stores data as **Objects**.

There are four important Git Objects.

Blob

Stores the content of a single file.

Example:

notes.txt

Each file becomes a Blob object.

Tree

Represents folders and directory structures.

Example:

Project


index.html


style.css


app.js

Commit Object

Stores:

- Snapshot of project
- Author
- Date
- Commit Message
- Parent Commit

Every commit creates a new Commit Object.

Tag Object

Tags are used to mark important versions.

Example:

Version 1.0

Version 2.0

Release Candidate

3.9 HEAD Pointer

HEAD is one of the most important concepts in Git.

HEAD points to the **current branch and latest commit**.

Example:

HEAD

↓

main

↓

Latest Commit

Whenever a new commit is created, HEAD automatically moves to it.

3.10 Git Lifecycle

Every file in Git passes through different states.

Explanation

- **Untracked** → Git is not monitoring the file.
- **Tracked** → Git is monitoring the file.
- **Modified** → File has changed.
- **Staged** → Ready for commit.
- **Committed** → Saved permanently in Git history.

3.11 Repository Initialization Workflow

3.12 Repository Storage Architecture

Developer

↓

Working Directory

↓

Staging Area

↓

Local Repository

↓

GitHub

3.13 Best Practices

- Keep one repository for one project.
- Commit frequently.
- Use meaningful commit messages.
- Push changes regularly.
- Never edit the `.git` folder manually.
- Organize project files clearly.

3.14 Common Mistakes

❌ Forgetting to initialize the repository.

❌ Deleting the `.git` folder accidentally.

❌ Confusing the Working Directory with the Staging Area.

❌ Forgetting to commit staged changes.

❌ Pushing incomplete work to the remote repository.

Real-Time Scenario

A developer is building an **Employee Management System**.

Workflow:

1. Create project folder.
2. Initialize Git.
3. Develop new features.
4. Stage changes.
5. Commit changes.
6. Push the repository to GitHub.
7. Collaborate with team members.

This ensures all project history is safely tracked and shared.

Interview Questions

1. What is a Git Repository?

Answer:

A Git Repository is a storage location that contains project files, commit history, branches, tags, and configuration required for version control.

---

2. What is the purpose of the `.git` directory?

Answer:

The `.git` directory stores all Git metadata, including commits, branches, configuration files, references, and repository history.

---

3. What is the difference between the Working Directory and the Staging Area?

Answer:

- The **Working Directory** contains files that are currently being edited.
- The **Staging Area** temporarily stores selected changes before they are committed to the repository.

---

4. What is HEAD in Git?

Answer:

HEAD is a pointer that references the current branch and the latest commit, helping Git determine the current working state of the repository.

---

5. What are Git Objects?

Answer:

Git stores data as objects:

- **Blob** → File content
- **Tree** → Directory structure
- **Commit** → Snapshot and metadata
- **Tag** → Named reference to important commits

Practical Lab

Task 1

Initialize a new Git repository using `git init`.

Task 2

Locate and inspect the hidden `.git` directory.

Task 3

Create three files and observe their state using `git status`.

Task 4

Draw the Git workflow showing the Working Directory, Staging Area, Local Repository, and Remote Repository.

Task 5

Explain the purpose of the four Git objects (Blob, Tree, Commit, Tag) with examples.

Module 4: Basic Git Commands

Learning Objectives

After completing this module, you will be able to:

- Understand the purpose of essential Git commands.
- Check repository status.
- Stage files for commit.
- Save changes using commits.
- View commit history.
- Compare file changes.
- Remove files from staging.
- Learn the Git command workflow.
- Apply Git commands in real-world development.

4.1 Introduction

Git commands are used to manage and track changes in a repository. Every Git project follows a workflow where files are created, modified, staged, committed, and shared.

Some of the most frequently used commands are:

- `git status`
- `git add`
- `git commit`
- `git log`
- `git diff`
- `git restore`
- `git rm`
- `git help`

Real-Time Example

A developer creates a new React project.

Tasks:

- Create project files.
- Check repository status.
- Stage modified files.
- Commit changes.
- View project history.
- Push changes to GitHub.

All these tasks are performed using basic Git commands.

4.2 Git Status (`git status`)

The `git status` command shows the current state of the repository.

Syntax

git status

Example

git status

Example Output

On branch main

No commits yet

Untracked files:

README.md

Purpose

- Shows modified files.
- Shows staged files.
- Shows untracked files.
- Displays current branch.

4.3 Git Add (`git add`)

The `git add` command moves changes from the **Working Directory** to the **Staging Area**.

Syntax

git add filename

Add Single File

git add index.html

Add Multiple Files

git add file1.txt file2.txt

Add All Files

git add .

4.4 Git Commit (`git commit`)

A commit permanently saves staged changes into the local repository.

Syntax

git commit -m "Commit Message"

Example

git commit -m "Added login page"

Example Output

[main abc1234]

Added login page

1 file changed

Good Commit Message Examples

- Added Login Page
- Fixed Navigation Bug
- Updated README
- Improved UI Design
- Added Payment Module

4.5 Git Log (`git log`)

Displays the commit history.

Syntax

git log

Example Output

commit 91fd56

Author: Prasanna

Date: July 2026

Added Login Feature

Useful Options

Compact Log

git log --oneline

Graph View

git log --graph

Limit Results

git log -5

4.6 Git Diff (`git diff`)

Shows differences between file versions.

Syntax

git diff

Example

git diff

Git highlights:

- Added lines
- Removed lines
- Modified lines

Compare Staged Changes

git diff --staged

4.7 Git Restore (`git restore`)

Used to discard unwanted changes.

Restore File

git restore index.html

This restores the file to its last committed version.

4.8 Git Remove (`git rm`)

Deletes files and stages the deletion.

Syntax

git rm filename

Example

git rm oldfile.txt

Commit the deletion:

git commit -m "Removed unused file"

4.9 Git Help

Git provides built-in documentation.

General Help

git help

Command Help

git help commit

or

git commit --help

4.10 Git Command Workflow

Create File

git status

git add

git commit

git log

4.11 Repository Lifecycle

Working Directory

git add

Staging Area

git commit

Local Repository

4.12 Common Git Commands Summary

Command	Purpose
`git status`	Check repository status
`git add`	Stage files
`git add .`	Stage all changes
`git commit -m`	Save changes
`git log`	View commit history
`git log --oneline`	Short commit history
`git diff`	Compare file changes
`git diff --staged`	Compare staged changes
`git restore`	Restore file
`git rm`	Delete tracked file
`git help`	Display documentation

4.13 Best Practices

- Check `git status` before committing.
- Write meaningful commit messages.
- Commit small and logical changes.
- Review changes using `git diff`.
- Avoid committing unnecessary files.
- Read command documentation using `git help`.

4.14 Common Mistakes

❌ Forgetting to stage files before committing.

❌ Using unclear commit messages such as "update" or "changes".

❌ Forgetting to review changes before committing.

❌ Accidentally deleting tracked files.

❌ Making one huge commit instead of several small commits.

Real-Time Scenario

A developer updates a company's website.

Workflow:

Check repository:

git status

Stage files:

git add .

Commit changes:

git commit -m "Updated homepage banner"

View history:

git log --oneline

Review modifications:

git diff

This is the workflow developers follow many times every day.

Interview Questions

1. What is the purpose of `git status`?

Answer:

`git status` displays the current state of the repository, including staged, modified, and untracked files, as well as the active branch.

---

2. What is the difference between `git add` and `git commit`?

Answer:

- `git add` moves changes to the **Staging Area**.
- `git commit` permanently records the staged changes in the local repository.

---

3. Why should commit messages be meaningful?

Answer:

Meaningful commit messages make it easier to understand the purpose of each change, review project history, and collaborate with team members.

---

4. What does `git diff` do?

Answer:

`git diff` compares changes between different versions of files, showing added, removed, and modified content.

---

5. What is the purpose of `git log --oneline`?

Answer:

`git log --oneline` displays a concise view of the commit history, showing one commit per line with a short commit hash and message.

Practical Lab

Task 1

Create a file named `README.md`.

Task 2

Check the repository status using:

git status

Task 3

Stage the file using:

git add README.md

Task 4

Commit the changes with the message:

Initial project setup

Task 5

View the commit history using:

git log --oneline

Module 5: Branching and Merging

Learning Objectives

After completing this module, you will be able to:

- Understand Git branches and their importance.
- Create, switch, rename, and delete branches.
- Merge branches into the main branch.
- Understand Fast-Forward and Three-Way Merge.
- Resolve merge conflicts.
- Learn professional branching strategies.
- Apply branching in real-world software development.

5.1 Introduction

In software development, multiple developers often work on different features simultaneously.

For example:

- Developer A works on the Login page.
- Developer B develops the Payment module.
- Developer C fixes bugs.
- Developer D improves the UI.

If everyone works directly on the **main** branch, changes may overwrite each other and create instability.

Git solves this problem using **branches**.

Definition

A **Branch** is an independent line of development that allows developers to work on features, bug fixes, or experiments without affecting the main project.

Real-Time Example

A banking application has separate teams working on:

- Login System
- Money Transfer
- Bill Payments
- Notifications

Each team develops its feature in a separate branch. Once testing is complete, the branches are merged into the **main** branch.

5.2 What is a Branch?

A branch is simply a pointer to a series of commits.

When a new branch is created:

- It starts from the current commit.
- Changes remain isolated.
- Other branches remain unaffected.

Example:

main
│
├── Login Branch
│
├── Payment Branch
│
└── Search Branch

5.3 Why Branches are Important

Advantages:

- Parallel development.
- Easy feature development.
- Safe experimentation.
- Faster collaboration.
- Simplified bug fixing.
- Better project organization.

5.4 View Existing Branches

Display all local branches.

Syntax

git branch

Example Output

* main

login

payment

The * indicates the current branch.

5.5 Create a New Branch

Syntax

git branch login

This creates a new branch named **login**.

The current branch remains unchanged.

5.6 Switch Branches

Move from one branch to another.

Syntax

git checkout login

Modern Git also supports:

git switch login

5.7 Create and Switch in One Command

git checkout -b payment

or

git switch -c payment

This creates the branch and switches to it immediately.

5.8 Rename a Branch

Syntax

git branch -m new-name

Example

git branch -m login authentication

5.9 Delete a Branch

Delete a merged branch.

Syntax

git branch -d login

Force delete:

git branch -D login

5.10 What is Merging?

Merging combines changes from one branch into another.

Example:

Login Branch

main Branch

After merging, all login feature changes become part of the main project.

5.11 Merge a Branch

Step 1: Switch to the destination branch.

git checkout main

Step 2: Merge another branch.

git merge login

Git combines the changes into the main branch.

5.12 Types of Merge

Fast-Forward Merge

Occurs when no new commits exist on the destination branch.

main
|
▼
A—B

login

A—B—C

↓

A—B—C

Git simply moves the branch pointer forward.

Three-Way Merge

Occurs when both branches contain different commits.

C
/

A—B

\

D

Git creates a new merge commit combining both histories.

5.13 Merge Conflicts

A merge conflict occurs when two branches modify the **same line** of the same file.

Example:

Branch A:

Welcome User

Branch B:

Welcome Customer

Git cannot determine which version to keep automatically.

Resolving Merge Conflicts

Steps:

1. Open the conflicted file.
2. Review Git conflict markers.
3. Keep the correct content.
4. Save the file.
5. Stage the file.

git add filename

6. Complete the merge.

git commit

5.14 Branching Workflow

main

Create Branch

Develop Feature

Commit Changes

Merge into main

Delete Branch

5.15 Branch Naming Conventions

Professional teams use meaningful branch names.

Examples:

feature/login

feature/payment

bugfix/navbar

hotfix/security

release/v1.0

Avoid names like:

branch1

test

abc

newbranch

5.16 Common Branch Commands

Command	Purpose
`git branch`	List branches
`git branch name`	Create branch
`git checkout name`	Switch branch
`git switch name`	Switch branch (modern)
`git checkout -b name`	Create and switch

`git switch -c name`	Create and switch (modern)
`git merge branch`	Merge branch
`git branch -d`	Delete merged branch
`git branch -D`	Force delete branch
`git branch -m`	Rename branch

5.17 Best Practices

- Create a separate branch for every feature.
- Keep branches focused on one task.
- Merge frequently to reduce conflicts.
- Delete merged branches.
- Use meaningful branch names.
- Test code before merging.

5.18 Common Mistakes

❌ Working directly on the `main` branch.

❌ Keeping long-lived branches without updates.

❌ Force deleting branches containing important work.

❌ Ignoring merge conflicts.

❌ Using meaningless branch names.

Real-Time Scenario

A company is developing an E-Commerce application.

Workflow:

1. Create a feature branch.

git checkout -b feature/cart

2. Develop the shopping cart feature.
3. Commit changes.

git commit -m "Added shopping cart"

4. Switch to the main branch.

git checkout main

5. Merge the feature.

git merge feature/cart

6. Delete the feature branch.

git branch -d feature/cart

This workflow is commonly followed in professional software development teams.

Interview Questions

1. What is a Git Branch?

Answer:

A Git branch is an independent line of development that allows developers to work on features or fixes without affecting the main branch.

---

2. Why are branches used?

Answer:

Branches allow parallel development, isolate new features, simplify collaboration, and reduce the risk of affecting stable code.

---

3. What is the difference between a Fast-Forward Merge and a Three-Way Merge?

Answer:

- **Fast-Forward Merge** simply moves the branch pointer when there are no diverging commits.
- **Three-Way Merge** creates a new merge commit when both branches contain independent changes.

---

4. What causes a merge conflict?

Answer:

A merge conflict occurs when two branches modify the same part of a file and Git cannot automatically determine which change should be kept.

---

5. How do you create and switch to a new branch in one command?

Answer:

git checkout -b feature-name

or (modern Git):

git switch -c feature-name

Practical Lab

Task 1

Create a new branch named `feature/profile`.

Task 2

Switch to the new branch.

Task 3

Make a small file change and commit it.

Task 4

Merge the `feature/profile` branch into the `main` branch.

Task 5

Delete the merged branch and verify it no longer appears in the branch list.

Module 6: GitHub Basics

Learning Objectives

After completing this module, you will be able to:

- Understand GitHub and its features.
- Create a GitHub account.
- Create a new repository.
- Connect a local Git repository to GitHub.
- Push local code to GitHub.
- Clone repositories.
- Fork repositories.
- Create and manage Pull Requests.
- Understand GitHub collaboration workflow.

6.1 Introduction

GitHub is a cloud-based platform built on Git that allows developers to host, manage, and collaborate on software projects.

Millions of developers and organizations use GitHub to:

- Store source code.
- Collaborate with teams.
- Review code.
- Track project issues.
- Contribute to open-source projects.
- Automate software workflows.

GitHub makes collaboration simple by providing a centralized platform while Git manages version control locally.

Definition

GitHub is a cloud-based hosting platform that uses Git to store, manage, and collaborate on software projects.

Real-Time Example

A software company develops an **Online Shopping Application**.

Workflow:

- Developers write code locally using Git.
- Changes are pushed to GitHub.
- Team members review the code.
- Approved changes are merged into the main branch.
- The latest version becomes available to everyone.

6.2 Features of GitHub

GitHub provides numerous features for software development.

Key Features

- Cloud Repository Hosting
- Version Control
- Team Collaboration
- Pull Requests
- Issues Tracking
- Project Management
- GitHub Actions (CI/CD)
- Releases
- Discussions
- Wiki Documentation

6.3 Creating a GitHub Account

Step 1

Visit:

https://github.com

Step 2

Click:

Sign Up

Step 3

Enter:

- Username
- Email Address
- Password

Step 4

Verify your email.

Step 5

Log in to GitHub.

6.4 GitHub Dashboard

After login, the dashboard provides access to:

- Repositories
- Organizations
- Projects
- Issues
- Pull Requests
- Notifications
- Profile

6.5 Creating a Repository

A repository stores your project files.

Steps

Click:

New Repository

Enter:

- Repository Name
- Description
- Visibility

Visibility Options:

- Public
- Private

Optionally initialize with:

- README
- .gitignore
- License

Click:

Create Repository

6.6 Local Repository vs GitHub Repository

Local Repository	GitHub Repository
Stored on your computer	Stored in the cloud
Works offline	Accessible online
Used for development	Used for collaboration

Managed by Git Hosted by GitHub

6.7 Connecting Local Repository to GitHub

After creating a repository on GitHub, connect your local project.

Add Remote Repository

git remote add origin https://github.com/username/project.git

Verify remote:

git remote -v

Example Output

origin

fetch

origin

push

6.8 Push Code to GitHub

Upload local commits to GitHub.

First Push

git push -u origin main

Later pushes:

git push

After pushing, the repository appears on GitHub.

6.9 Clone Repository

Download an existing GitHub repository.

Syntax

git clone https://github.com/username/project.git

Example Output

Cloning into 'project'...

Git automatically creates the project folder and downloads all files.

6.10 Fork Repository

A **Fork** creates a personal copy of another user's repository.

Purpose:

- Contribute to open-source projects.
- Experiment independently.
- Submit Pull Requests.

Example:

Original Repository

↓

Fork

↓

Your GitHub Account

6.11 Pull Requests (PR)

A **Pull Request (PR)** is a request to merge changes from one branch into another.

Workflow:

Feature Branch

↓

Push to GitHub

↓

Create Pull Request

↓

Code Review

↓

Merge

Benefits:

- Code review.
- Team discussion.
- Automated testing.
- Quality assurance.

6.12 GitHub Collaboration Workflow

Developer

↓

Local Repository

↓

GitHub Repository

↓

Pull Request

↓

Review

↓

Merge

6.13 Common GitHub Commands

Command	Purpose
`git remote add origin`	Connect remote repository
`git remote -v`	View remote repositories
`git push`	Upload commits
`git clone`	Download repository
`git pull`	Download latest changes
`git fetch`	Retrieve updates without merging

6.14 Best Practices

- Use descriptive repository names.
- Write a clear README file.
- Keep repositories organized.
- Commit changes frequently.
- Push changes regularly.
- Review Pull Requests before merging.
- Protect the `main` branch.
- Use `.gitignore` for unnecessary files.

6.15 Common Mistakes

❌ Forgetting to connect the remote repository.

❌ Pushing directly to the `main` branch without review.

❌ Keeping sensitive files (API keys, passwords) in the repository.

❌ Ignoring Pull Requests.

❌ Using unclear repository names.

Real-Time Scenario

A developer finishes building a React application.

Steps:

Initialize Git:

git init

Connect to GitHub:

git remote add origin https://github.com/company/react-app.git

Push the project:

git push -u origin main

Another developer clones the project:

git clone https://github.com/company/react-app.git

A new feature is developed in a separate branch, pushed to GitHub, and merged using a Pull Request.

Interview Questions

1. What is GitHub?

Answer:

GitHub is a cloud-based platform that hosts Git repositories and provides tools for version control, collaboration, code review, and project management.

---

2. What is the difference between `git clone` and `git fork`?

Answer:

- `git clone` creates a local copy of a repository on your computer.
- **Forking** creates a copy of someone else's repository under your own GitHub account, allowing independent development.

---

3. What is a Pull Request?

Answer:

A Pull Request is a request to merge changes from one branch or fork into another branch after code review and approval.

---

4. Why is `git remote add origin` used?

Answer:

It links a local Git repository to a remote GitHub repository, enabling commands such as `git push` and `git pull`.

---

5. What is the purpose of `git push -u origin main`?

Answer:

It uploads the local `main` branch to the remote repository and sets the upstream branch, allowing future pushes with just `git push`.

Practical Lab

Task 1

Create a GitHub account.

Task 2

Create a new public repository named `GitPractice`.

Task 3

Connect your local repository to GitHub using `git remote add origin`.

Task 4

Push your first commit to GitHub.

Task 5

Clone your repository into a different folder and verify that all files are downloaded.

Module 7: Remote Repository Management

Learning Objectives

After completing this module, you will be able to:

- Understand Remote Repositories.
- Add, view, rename, and remove remote repositories.
- Push and pull changes.
- Fetch updates from remote repositories.
- Understand upstream branches.
- Synchronize local and remote repositories.
- Manage multiple remote repositories.
- Apply remote repository workflows in real-world development.

7.1 Introduction

A **Remote Repository** is a Git repository stored on a remote server, such as GitHub, GitLab, or Bitbucket. It enables multiple developers to collaborate on the same project.

Remote repositories help teams:

- Share source code.
- Synchronize changes.
- Collaborate efficiently.
- Maintain centralized project backups.

Definition

A **Remote Repository** is a Git repository hosted on a remote server that allows multiple developers to share, synchronize, and collaborate on project code.

Real-Time Example

A software company has developers working from different locations.

Workflow:

- Developer A writes new code.
- Developer B fixes bugs.
- Developer C reviews code.

All developers synchronize their work through a GitHub remote repository.

7.2 Local vs Remote Repository

Local Repository	Remote Repository
Stored on local machine	Stored on GitHub or another server
Works offline	Requires internet access
Used for development	Used for collaboration
Private to the developer	Shared with the team

7.3 Viewing Remote Repositories

Display configured remote repositories.

Syntax

git remote

Display detailed information:

git remote -v

Example Output

origin https://github.com/user/project.git (fetch)

origin https://github.com/user/project.git (push)

7.4 Adding a Remote Repository

Connect a local repository to GitHub.

Syntax

git remote add origin https://github.com/username/project.git

Verify:

git remote -v

7.5 Renaming a Remote Repository

Sometimes a remote name needs to be changed.

Syntax

git remote rename origin upstream

Verify:

git remote

7.6 Removing a Remote Repository

Delete a configured remote.

Syntax

git remote remove origin

Verify:

git remote

7.7 Push Changes (`git push`)

The `git push` command uploads local commits to the remote repository.

Syntax

git push origin main

If the upstream branch is already configured:

git push

7.8 Pull Changes (`git pull`)

The `git pull` command downloads and merges changes from the remote repository.

Syntax

git pull origin main

Workflow:

Remote Repository

Download Changes

Merge into Local Branch

7.9 Fetch Changes (`git fetch`)

`git fetch` downloads updates **without merging** them.

Syntax

git fetch origin

Difference:

- `git fetch` → Downloads changes only.
- `git pull` → Downloads and merges changes.

7.10 Push vs Pull vs Fetch

Command	Purpose
`git push`	Upload local commits
`git pull`	Download and merge changes
`git fetch`	Download changes only

7.11 Upstream Branch

An **upstream branch** links a local branch with a remote branch.

Set Upstream

git push -u origin main

Benefits:

- Future pushes only require:

git push

Future pulls only require:

git pull

7.12 Managing Multiple Remotes

A project can have more than one remote repository.

Example:

origin

↓

GitHub

upstream

↓

Company Repository

View all remotes:

git remote -v

Push to a specific remote:

git push upstream main

7.13 Remote Repository Workflow

Local Repository

git push

GitHub

Developer B

git pull

7.14 Synchronization Workflow

Developer A

Push

GitHub

Developer B

7.15 Common Remote Commands

Command	Purpose
`git remote`	View remotes
`git remote -v`	View remote URLs
`git remote add`	Add remote
`git remote rename`	Rename remote
`git remote remove`	Remove remote
`git push`	Upload commits
`git pull`	Download and merge
`git fetch`	Download only
`git push -u`	Set upstream branch

7.16 Best Practices

- Pull the latest changes before starting work.
- Push commits regularly.
- Verify remote URLs before pushing.
- Use meaningful remote names.
- Review fetched changes before merging.
- Keep local and remote repositories synchronized.

7.17 Common Mistakes

❌ Forgetting to pull before pushing.

❌ Pushing to the wrong remote.

❌ Removing the wrong remote repository.

❌ Ignoring merge conflicts after `git pull`.

❌ Forgetting to configure the upstream branch.

Real-Time Scenario

A development team is working on a web application.

Developer A:

git add .
git commit -m "Added dashboard"
git push origin main

Developer B updates the local repository:

git fetch origin

Then merges the changes:

git pull origin main

The team stays synchronized while developing the project.

Interview Questions

1. What is a Remote Repository?

Answer:

A Remote Repository is a Git repository hosted on a remote server that enables developers to share code and collaborate on projects.

---

2. What is the difference between `git fetch` and `git pull`?

Answer:

- `git fetch` downloads updates from the remote repository without merging them.
- `git pull` downloads updates and automatically merges them into the current branch.

---

3. Why is `git push -u origin main` used?

Answer:

It uploads the local `main` branch to the remote repository and sets the upstream branch, allowing future `git push` and `git pull` commands without specifying the remote and branch.

---

4. How do you view configured remote repositories?

Answer:

Use:

git remote -v

This displays all configured remotes along with their fetch and push URLs.

---

5. Can a Git repository have multiple remotes?

Answer:

Yes. A Git repository can have multiple remotes, such as an `origin` for your personal GitHub repository and an `upstream` for the original project repository.

Practical Lab

Task 1

Add a remote GitHub repository to your local project.

Task 2

Verify the configured remote using:

git remote -v

Task 3

Push your local repository to GitHub.

Task 4

Use `git fetch` to retrieve updates without merging them.

Task 5

Configure an upstream branch and test simple `git push` and `git pull` commands.

Module 8: Git Collaboration

Learning Objectives

After completing this module, you will be able to:

- Understand collaborative software development.
- Learn team-based Git workflows.
- Understand Pull Requests in detail.
- Learn the Code Review process.
- Understand Merge Requests and approval workflows.
- Resolve collaboration conflicts.
- Follow Git collaboration best practices.
- Apply professional collaboration techniques in real-world projects.

8.1 Introduction

Modern software development is rarely done by a single developer. Teams of developers work together on the same project using Git and GitHub.

Git Collaboration allows developers to:

- Work independently on features.
- Share code with teammates.
- Review each other's work.
- Merge approved changes safely.
- Maintain code quality.

GitHub provides collaboration features such as Pull Requests, Code Reviews, Issues, Discussions, and Branch Protection.

Definition

Git Collaboration is the process of multiple developers working together on the same project using Git and GitHub while maintaining version control, code quality, and project history.

Real-Time Example

A software company develops a Food Delivery Application.

Team Members:

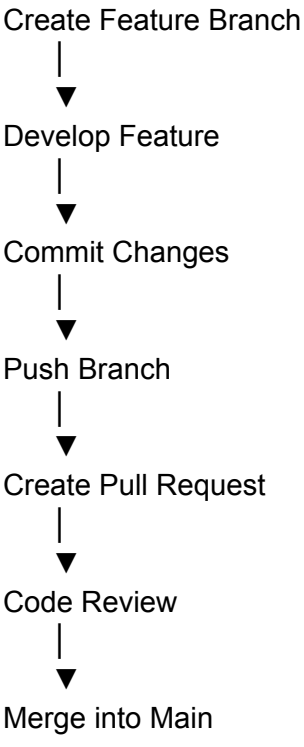
- Developer A → Login Module
- Developer B → Payment Module
- Developer C → Search Feature
- Developer D → Bug Fixes

Each developer works in a separate branch, submits a Pull Request, receives code review feedback, and merges changes after approval.

8.2 Team Collaboration Workflow

Professional teams usually follow this workflow.

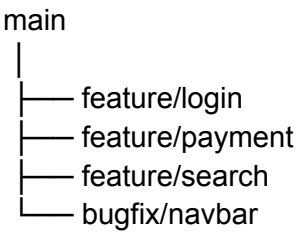
Clone Repository



8.3 Feature Branch Workflow

Instead of working directly on the **main** branch, developers create a new feature branch.

Example:



Benefits:

- Prevents unstable code from reaching the main branch.
- Makes feature development independent.
- Simplifies testing and code review.

8.4 Pull Requests (PR)

A **Pull Request (PR)** is a request to merge changes from one branch into another.

Workflow:

Feature Branch

Push to GitHub

Create Pull Request

Code Review

Merge

A Pull Request includes:

- Title
- Description
- Changed files
- Commit history
- Comments
- Review status

8.5 Code Review

Before merging code, experienced developers review it.

The reviewer checks:

- Code quality
- Readability
- Logic
- Performance
- Security
- Coding standards
- Test coverage

Possible review outcomes:

- ✅ Approved
- 🔄 Changes Requested
- 💬 Comment Added

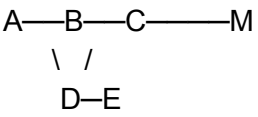
8.6 Merge Requests

After approval, the Pull Request can be merged.

Common merge strategies:

Merge Commit

Creates a separate merge commit.



Squash Merge

Combines multiple commits into a single commit.

Before:

A

B

C

D

After:

A

Single Commit

Advantages:

- Cleaner commit history.
- Easier maintenance.

Rebase and Merge

Rewrites commit history before merging.

Advantages:

- Linear history.

- Cleaner repository.

8.7 Collaboration Conflict Resolution

Sometimes multiple developers edit the same file.

Example:

Developer A:

Welcome User

Developer B:

Welcome Customer

Git cannot automatically choose one version.

Resolution Steps:

1. Pull latest changes.
2. Open conflicted file.
3. Edit manually.
4. Remove conflict markers.
5. Save the file.
6. Stage changes.

git add filename

7. Commit merge.

git commit

8.8 GitHub Collaboration Tools

GitHub offers additional collaboration features.

Issues

Used for:

- Bug tracking
- Feature requests
- Task management

Discussions

Used for:

- Team communication
- Project planning
- Questions

Projects

Kanban-style boards used to organize work.

Example:

To Do

↓

In Progress

↓

Review

↓

Done

Wiki

Used to maintain project documentation.

8.9 Collaboration Best Practices

- Create one branch per feature.
- Keep Pull Requests small.
- Write meaningful commit messages.
- Review code before merging.
- Pull latest changes frequently.
- Resolve conflicts immediately.

- Never push incomplete code to the main branch.

8.10 Professional Git Workflow

Clone Repository

Create Branch

Develop Feature

Commit

Push

Pull Request

Code Review

Merge

8.11 Common Collaboration Commands

Command	Purpose
`git clone`	Download repository
`git branch`	Create/List branches
`git checkout`	Switch branches
`git add`	Stage changes

`git commit`	Save changes
`git push`	Upload branch
`git pull`	Download latest changes
`git fetch`	Retrieve updates
`git merge`	Merge branches

8.12 Best Practices

- Keep branches short-lived.
- Write descriptive Pull Request titles.
- Review every Pull Request.
- Follow coding standards.
- Test before creating a Pull Request.
- Merge only approved code.
- Delete merged branches.

8.13 Common Mistakes

❌ Working directly on the `main` branch.

❌ Creating very large Pull Requests.

❌ Ignoring code review comments.

❌ Forgetting to pull the latest changes.

❌ Leaving merge conflicts unresolved.

❌ Pushing unfinished features.

Real-Time Scenario

A team is building an **Online Banking System**.

Developer A:

git checkout -b feature/login

After development:

git add .
git commit -m "Added login module"
git push origin feature/login

A Pull Request is created.

Reviewer approves the changes.

The feature is merged into the `main` branch.

The branch is then deleted.

This workflow is followed for every feature developed by the team.

Interview Questions

1. What is Git Collaboration?

Answer:

Git Collaboration is the process of multiple developers working together on the same project using Git and GitHub while maintaining version control, code quality, and project history.

---

2. What is a Pull Request?

Answer:

A Pull Request is a request to merge changes from one branch into another after code review and approval.

---

3. Why is Code Review important?

Answer:

Code Review helps identify bugs, improve code quality, ensure coding standards are followed, and share knowledge among team members before changes are merged.

---

4. Why should developers use feature branches?

Answer:

Feature branches isolate development work, reduce the risk of affecting stable code, simplify testing, and make collaboration easier.

---

5. What are common merge strategies?

Answer:

The three common merge strategies are:

- **Merge Commit**
- **Squash Merge**
- **Rebase and Merge**

Each strategy has different effects on the project's commit history.

Practical Lab

Task 1

Clone a GitHub repository.

Task 2

Create a feature branch named `feature/profile`.

Task 3

Make changes, commit them, and push the branch to GitHub.

Task 4

Create a Pull Request from `feature/profile` to `main`.

Task 5

Review the Pull Request, merge it, and delete the feature branch.

Module 9: Advanced Git Commands

Learning Objectives

After completing this module, you will be able to:

- Understand advanced Git operations.
- Temporarily save work using `git stash`.
- Undo changes using `git reset`.
- Safely reverse commits using `git revert`.
- Apply specific commits using `git cherry-pick`.
- Rewrite commit history using `git rebase`.
- Create and manage Git tags.
- Apply advanced Git workflows in real-world projects.

9.1 Introduction

As software projects grow, developers often need to:

- Switch tasks without losing work.
- Undo accidental commits.
- Restore previous project versions.
- Reuse commits from another branch.
- Organize release versions.
- Maintain a clean commit history.

Git provides several advanced commands to handle these situations efficiently.

Definition

Advanced Git Commands are specialized commands used to manage repository history, recover changes, organize releases, and improve collaboration in professional software development.

Real-Time Example

A developer is working on a **Payment Module** but receives an urgent request to fix a production bug.

Instead of committing incomplete work, they use:

git stash

Later, after fixing the bug, they restore their previous work using:

git stash pop

9.2 Git Stash

`git stash` temporarily saves uncommitted changes without creating a commit.

Syntax

git stash

Workflow

Modified Files

git stash

Temporary Storage

View all stashes:

git stash list

Restore the latest stash:

git stash pop

Apply without deleting:

git stash apply

Delete a stash:

git stash drop

9.3 Git Reset

`git reset` moves the current branch to a previous commit.

Types of Reset

Soft Reset

Keeps staged changes.

git reset --soft HEAD~1

Mixed Reset (Default)

Keeps files but removes them from the staging area.

git reset HEAD~1

Hard Reset

Removes commits, staged changes, and working directory changes.

git reset --hard HEAD~1

⚠️ **Warning:** `--hard` permanently discards uncommitted work.

9.4 Git Revert

`git revert` creates a new commit that reverses the changes of a previous commit.

Syntax

git revert commit_id

Example:

git revert a1b2c3d

Unlike `git reset`, `git revert` preserves project history and is safer for shared repositories.

9.5 Git Reset vs Git Revert

Git Reset	Git Revert
Removes commit history	Preserves history
Used for local changes	Safe for shared repositories
Can rewrite history	Creates a new reversing commit
Risky on public branches	Recommended for team projects

9.6 Git Cherry-Pick

`git cherry-pick` copies a specific commit from one branch to another.

Syntax

git cherry-pick commit_id

Example:

git cherry-pick 5d3f2ab

Real-Time Example

A bug fix exists in the `bugfix` branch.

Instead of merging the entire branch, only the required commit is copied into the `main` branch.

9.7 Git Rebase

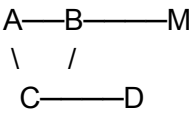
`git rebase` moves or reapplies commits on top of another branch.

Syntax

git rebase main

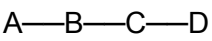
Merge vs Rebase

Merge



Creates a merge commit.

Rebase



Creates a clean, linear history.

Advantages

- Cleaner commit history.
- Easier project navigation.
- Better for feature branches.

9.8 Git Tag

Tags mark important project versions.

Examples:

- Version 1.0
- Version 2.0
- Production Release

Create Lightweight Tag

git tag v1.0

Create Annotated Tag

git tag -a v1.0 -m "First Stable Release"

View Tags

git tag

Push Tags

git push origin v1.0

Push all tags:

git push origin --tags

9.9 Git Stash Workflow

Modify Files

git stash

Temporary Storage

git stash pop

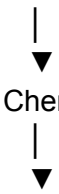
Continue Work

9.10 Git History Management

Commit

Reset

Revert



9.11 Advanced Git Commands Summary

Command	Purpose
`git stash`	Save temporary changes
`git stash pop`	Restore latest stash
`git stash apply`	Apply stash without removing it
`git reset`	Move to a previous commit
`git revert`	Reverse a commit safely
`git cherry-pick`	Copy a specific commit
`git rebase`	Rewrite commit history
`git tag`	Create version tags
`git push --tags`	Push tags to remote

9.12 Best Practices

- Use `git stash` before switching tasks.
- Prefer `git revert` over `git reset` for shared repositories.
- Use `git rebase` to maintain a clean history before merging.
- Create tags for important releases.
- Test thoroughly after using `git cherry-pick`.
- Avoid rewriting the history of public branches.

9.13 Common Mistakes

❌ Using `git reset --hard` without understanding its impact.

❌ Rebasing branches already shared with the team.

❌ Forgetting to push release tags.

❌ Applying the wrong stash.

❌ Cherry-picking the wrong commit.

Real-Time Scenario

A developer is working on a shopping cart feature.

1. Save unfinished work:

git stash

2. Switch to the `main` branch.
3. Fix a production bug.
4. Commit the bug fix.
5. Return to the feature branch.
6. Restore saved work:

git stash pop

7. Create a release tag:

git tag -a v2.0 -m "Shopping Cart Release"

8. Push the tag:

git push origin --tags

This workflow is commonly used in professional software development.

Interview Questions

1. What is the purpose of `git stash`?

Answer:

`git stash` temporarily saves uncommitted changes so you can switch branches or work on another task without losing your progress.

---

2. What is the difference between `git reset` and `git revert`?

Answer:

- `git reset` rewrites commit history and is mainly used for local changes.
- `git revert` creates a new commit that reverses an earlier commit while preserving project history, making it safer for shared repositories.

---

3. When should `git cherry-pick` be used?

Answer:

`git cherry-pick` is used when you want to copy a specific commit from one branch to another without merging the entire branch.

---

4. Why is `git rebase` used?

Answer:

`git rebase` creates a cleaner, linear commit history by replaying commits on top of another branch.

---

5. What is a Git Tag?

Answer:

A Git Tag is a reference used to mark important points in a repository's history, such as software releases (e.g., `v1.0`, `v2.0`).

Practical Lab

Task 1

Create a temporary change and save it using `git stash`.

Task 2

Restore the saved work using `git stash pop`.

Task 3

Create an annotated tag named `v1.0`.

Task 4

Use `git cherry-pick` to apply a commit from another branch.

Task 5

Compare the output of `git merge` and `git rebase` using a sample repository.

Module 10: Git Internals

Learning Objectives

After completing this module, you will be able to:

- Understand Git's internal architecture.
- Learn how Git stores data.
- Understand SHA-1 hashing.
- Learn Git Objects (Blob, Tree, Commit, Tag).
- Understand the HEAD pointer.
- Learn Git References (Refs).
- Understand the Git Index (Staging Area).
- Apply Git Internals knowledge in troubleshooting and advanced workflows.

10.1 Introduction

Git is more than just a version control tool—it is a **content-addressable database**. Instead of storing changes as simple file differences, Git stores **snapshots** of your project and identifies them using unique cryptographic hashes.

Every commit, file, directory, and tag is stored as an object inside the `.git` directory.

Understanding Git Internals helps developers:

- Debug repository issues.
- Understand commit history.
- Recover lost commits.
- Learn how Git works behind the scenes.
- Perform advanced Git operations confidently.

Definition

Git Internals refers to the internal data structures and mechanisms Git uses to store project history, manage branches, and track changes.

Real-Time Example

When a developer commits code:

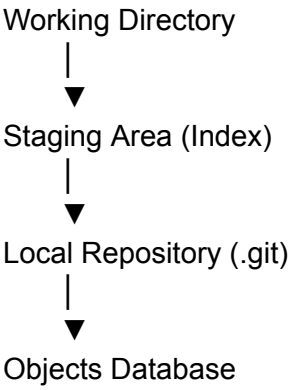
git commit -m "Added Login Module"

Git performs several actions:

- Creates Blob objects for file contents.
- Creates Tree objects for directories.
- Creates a Commit object.
- Generates a unique SHA-1 hash.
- Updates the HEAD pointer.

All of this happens automatically.

10.2 Git Internal Architecture



The `.git` directory contains all internal repository information.

10.3 SHA-1 Hash

Git uniquely identifies every object using a **SHA-1 (Secure Hash Algorithm 1)** hash.

Example:

9fceb02d0ae598e95dc970b74767f19372d61af8

Properties:

- 40 hexadecimal characters.
- Unique identifier.
- Generated based on object content.
- Changes whenever the content changes.

Advantages

- Ensures data integrity.
- Detects corruption.
- Uniquely identifies objects.

10.4 Git Objects

Git stores information as four main object types.

Blob

↓

Tree

↓

Commit

↓

Tag

Blob Object

A **Blob (Binary Large Object)** stores the content of a single file.

Example:

README.md

Each file becomes a separate Blob object.

Blob stores:

- File content only.
- No filename.
- No directory information.

Tree Object

A Tree object represents directories and file structure.

Example:

Project

├── README.md

├── src

└── package.json

Tree stores:

- Folder names.
- File names.
- References to Blob objects.
- References to other Tree objects.

Commit Object

A Commit object stores a snapshot of the project.

Each commit contains:

- Author
- Committer
- Commit Message
- Timestamp
- Parent Commit
- Tree Object Reference

Example:

Commit

↓

Tree

↓

Files

Tag Object

A Tag points to a specific commit.

Example:

v1.0

v2.0

Production Release

Tags are commonly used to mark software releases.

10.5 Git Object Relationship

Commit

|

▼

Tree

|

▼

Blob

Relationship:

- Commit references Tree.
- Tree references Blobs.
- Blob stores file contents.

10.6 HEAD Pointer

HEAD is one of the most important concepts in Git.

HEAD points to:

- Current branch.
- Latest commit.

Example:

HEAD

↓

main

↓

Latest Commit

Whenever a new commit is created:

- HEAD automatically moves to the latest commit.

10.7 Git References (Refs)

Git stores branch information using **References (Refs)**.

Types:

- Branch References
- Tag References
- Remote References

Example:

refs

├── heads

├── tags

└── remotes

Purpose:

- Track branches.
- Track tags.
- Track remote branches.

10.8 Git Index (Staging Area)

The **Index** (Staging Area) temporarily stores changes before committing.

Workflow:

Working Directory

↓

Index

↓

Commit

Purpose:

- Select files.
- Review changes.
- Prepare commits.

10.9 Git Object Storage

Objects are stored inside:

.git/objects/

Git compresses object data to reduce storage usage.

Objects remain unchanged once created.

10.10 Git Internal Workflow

Edit File

↓

Blob

↓

Tree

↓

Commit

↓

SHA-1 Hash

↓

HEAD Updated

10.11 Git Data Flow

Working Directory

↓

git add

↓

Index

↓

git commit

↓

Object Database

↓

GitHub

10.12 Important Git Internal Components

Component	Purpose
Blob	Stores file contents
Tree	Stores directory structure
Commit	Stores project snapshot
Tag	Marks important versions
HEAD	Points to current commit
Refs	Stores branch/tag references
Index	Temporary staging area
SHA-1	Unique object identifier

10.13 Best Practices

- Avoid modifying the `.git` directory manually.
- Understand Git Internals before using advanced commands.
- Create meaningful commits.
- Use tags for releases.
- Keep repository history clean.

10.14 Common Mistakes

❌ Deleting the `.git` directory.

❌ Editing Git object files manually.

❌ Confusing Blob objects with actual files.

❌ Misunderstanding the HEAD pointer.

❌ Deleting references accidentally.

Real-Time Scenario

A DevOps engineer accidentally deletes a branch.

Using knowledge of:

- Commit objects
- SHA-1 hashes
- References
- HEAD

the engineer locates the missing commit and restores the branch without losing project history.

Understanding Git Internals makes advanced recovery possible.

Interview Questions

1. What is SHA-1 in Git?

Answer:

SHA-1 is a cryptographic hashing algorithm used by Git to generate a unique 40-character identifier for every object stored in the repository.

---

2. What are the four Git Objects?

Answer:

Git stores data using four object types:

- Blob
- Tree
- Commit
- Tag

---

3. What is the purpose of the HEAD pointer?

Answer:

HEAD is a reference that points to the current branch and its latest commit, indicating the repository's current working state.

---

4. What is the Git Index?

Answer:

The Git Index (Staging Area) is a temporary storage area where selected changes are prepared before they are committed to the repository.

---

5. What is the difference between a Blob and a Tree object?

Answer:

- A **Blob** stores the contents of a file.
- A **Tree** stores the directory structure and references to Blob and other Tree objects.

Practical Lab

Task 1

Initialize a Git repository and locate the hidden `.git` directory.

Task 2

Create a file, commit it, and observe the repository structure.

Task 3

Create an annotated tag named `v1.0`.

Task 4

Draw the relationship between Blob, Tree, and Commit objects.

Task 5

Explain the purpose of HEAD, Refs, and the Index in your own words.

Module 11: GitHub Features

Learning Objectives

After completing this module, you will be able to:

- Understand the major features of GitHub.
- Learn GitHub Issues for bug tracking.
- Manage work using GitHub Projects.
- Use GitHub Discussions for collaboration.
- Create documentation with GitHub Wiki.
- Manage software releases.
- Host websites using GitHub Pages.
- Understand the basics of GitHub Actions.
- Apply GitHub features in real-world software development.

11.1 Introduction

GitHub is much more than a Git repository hosting service. It provides a complete ecosystem for software development, including project planning, issue tracking, documentation, automation, and deployment.

Organizations use GitHub to:

- Manage source code.
- Track bugs.
- Plan projects.
- Review code.
- Automate builds and deployments.
- Publish documentation.
- Collaborate with distributed teams.

Definition

GitHub Features are built-in tools that help developers collaborate, manage projects, automate workflows, document software, and publish applications efficiently.

Real-Time Example

A company develops an **E-Commerce Website**.

They use:

- **Repositories** → Store source code.
- **Issues** → Track bugs.
- **Projects** → Manage development tasks.
- **Pull Requests** → Review code.
- **Wiki** → Store technical documentation.
- **Actions** → Automatically test and deploy the application.
- **Releases** → Publish stable versions.

11.2 GitHub Issues

GitHub Issues help teams track:

- Bugs
- Feature requests
- Improvements
- Tasks
- Questions

Each issue contains:

- Title
- Description
- Labels
- Assignees
- Milestones
- Comments

Example:

Issue #25

Title:
Login button not working

Status:
Open

Assigned To:
Developer A

11.3 GitHub Projects

GitHub Projects provide a **Kanban-style project management board**.

Example Workflow:

To Do

In Progress

Code Review

Testing

Done

Benefits:

- Organize tasks.
- Assign team members.
- Track project progress.
- Improve team collaboration.

11.4 GitHub Discussions

GitHub Discussions provide a space for team communication.

Common Uses:

- Ask technical questions.
- Discuss new features.
- Share ideas.
- Announce updates.
- Community support.

Example:

Topic:
Should we migrate to React 19?

Replies:
15

11.5 GitHub Wiki

A Wiki is used for project documentation.

Typical Wiki Pages:

- Installation Guide
- API Documentation
- User Manual
- Developer Guide
- FAQs
- Troubleshooting

Benefits:

- Centralized documentation.
- Easy collaboration.
- Version-controlled documentation.

11.6 GitHub Releases

Releases are used to publish stable versions of software.

Example:

Version:
v1.0.0

Release Name:
Initial Stable Release

A release usually includes:

- Version number
- Release notes
- Downloadable files
- Changelog

11.7 GitHub Pages

GitHub Pages allows developers to host static websites directly from a GitHub repository.

Supported Content:

- HTML
- CSS
- JavaScript
- Documentation sites
- Portfolio websites

Example:

https://username.github.io/project

Common Uses:

- Personal portfolio
- Project documentation
- Blogs
- Landing pages

11.8 GitHub Actions

GitHub Actions is GitHub's built-in **CI/CD (Continuous Integration and Continuous Deployment)** platform.

It automates software workflows.

Common Tasks:

- Build applications.
- Run automated tests.
- Deploy applications.
- Check code quality.
- Send notifications.

Workflow:

Push Code

GitHub Actions

Build

Test

Deploy

11.9 GitHub Notifications

Notifications keep developers informed about:

- Pull Requests
- Issues
- Reviews

- Mentions
- Discussions
- Repository activity

Benefits:

- Stay updated.
- Respond quickly.
- Improve collaboration.

11.10 GitHub Insights

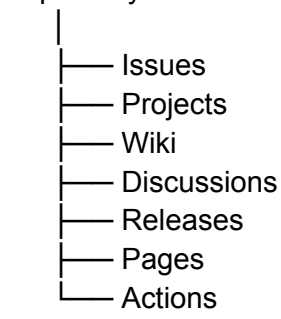
GitHub Insights provides analytics for repositories.

Information includes:

- Commit activity
- Contributor statistics
- Traffic
- Pull Request trends
- Code frequency

Managers use Insights to monitor project health and team productivity.

11.11 GitHub Feature Architecture

Repository


11.12 GitHub Features Summary

Feature	Purpose

Repository	Store source code
Issues	Track bugs and tasks
Projects	Manage work using Kanban boards
Discussions	Team communication
Wiki	Project documentation
Releases	Publish software versions
Pages	Host static websites
Actions	Automate build, test, and deployment
Notifications	Track repository updates
Insights	Analyze repository activity

11.13 Best Practices

- Use Issues for every bug and feature request.
- Keep Projects updated.
- Maintain detailed Wiki documentation.
- Create Releases for stable software versions.
- Automate repetitive tasks using GitHub Actions.
- Monitor repository Insights regularly.
- Enable notifications for important repositories.

11.14 Common Mistakes

❌ Tracking bugs outside GitHub Issues.

❌ Ignoring Pull Request reviews.

❌ Keeping documentation outdated.

❌ Publishing releases without release notes.

❌ Not automating repetitive workflows.

❌ Forgetting to update project boards.

Real-Time Scenario

A software company develops an **Online Banking Application**.

Workflow:

1. A bug is reported using GitHub Issues.
2. The issue is assigned to a developer.
3. The task is added to GitHub Projects.
4. The developer fixes the issue in a feature branch.
5. A Pull Request is created.
6. Team members review the code.
7. GitHub Actions runs automated tests.
8. The Pull Request is merged.
9. A new release (`v2.1.0`) is published.
10. Documentation is updated in the Wiki.

This end-to-end workflow is commonly used in enterprise software development.

Interview Questions

1. What is GitHub Issues?

Answer:

GitHub Issues is a feature used to track bugs, feature requests, tasks, and project-related discussions.

---

2. What is the purpose of GitHub Projects?

Answer:

GitHub Projects helps teams organize and manage work using Kanban-style boards, making it easier to track progress and assign tasks.

---

3. What is GitHub Actions?

Answer:

GitHub Actions is GitHub's built-in CI/CD platform that automates software development workflows such as building, testing, and deploying applications.

---

4. What is GitHub Pages?

Answer:

GitHub Pages is a hosting service that allows developers to publish static websites directly from a GitHub repository.

---

5. Why are GitHub Releases important?

Answer:

GitHub Releases allow developers to publish stable software versions, provide release notes, distribute build files, and maintain version history.

Practical Lab

Task 1

Create a GitHub repository and enable the **Issues** feature.

Task 2

Create three Issues:

- Login Bug
- Payment Error
- UI Improvement

Task 3

Create a GitHub Project board with the columns:

- To Do
- In Progress
- Review

- Done

Task 4

Create a Wiki page named **Installation Guide**.

Task 5

Create a release named **v1.0.0** with release notes

Module 12: Git Best Practices

Learning Objectives

After completing this module, you will be able to:

- Understand professional Git workflows.
- Write meaningful commit messages.
- Follow branch naming conventions.
- Use `.gitignore` effectively.
- Organize repositories professionally.
- Protect sensitive information.
- Learn security best practices.
- Apply Git standards used in enterprise software development.

12.1 Introduction

Writing code is only one part of software development. Professional developers also follow coding and version control standards to ensure projects remain organized, secure, and easy to maintain.

Git Best Practices help teams:

- Maintain clean repositories.
- Improve collaboration.
- Reduce merge conflicts.
- Protect sensitive data.
- Make project history easier to understand.
- Improve software quality.

Definition

Git Best Practices are a set of recommended guidelines and standards that help developers use Git efficiently, securely, and collaboratively.

Real-Time Example

A software company has 50 developers working on the same project.

Without Git standards:

- Random branch names.
- Poor commit messages.
- Sensitive files committed.
- Large merge conflicts.

With Git Best Practices:

- Clear commit history.
- Organized branches.
- Secure repositories.
- Easier collaboration.

12.2 Writing Good Commit Messages

A commit message should clearly describe the purpose of the change.

Good Examples

Added user authentication

Fixed payment gateway timeout

Updated API documentation

Improved dashboard UI

Removed unused CSS files

Poor Examples

Update

Fix

Changes

Done

Test

Recommended Commit Format

Short Summary

(Optional Detailed Description)

Example:

Added password reset feature

Implemented email verification and validation.

12.3 Branch Naming Conventions

Professional branch names improve readability.

Feature Branch

feature/login

feature/payment

feature/dashboard

Bug Fix Branch

bugfix/navbar

bugfix/login-error

Hotfix Branch

hotfix/security

hotfix/payment

Release Branch

release/v1.0

release/v2.1

Avoid names such as:

test

new

abc

branch1

12.4 Using `.gitignore`

`.gitignore` tells Git which files or folders should **not** be tracked.

Example:

node_modules/

.env

*.log

dist/

build/

Benefits:

- Keeps repositories clean.
- Prevents unnecessary files from being committed.
- Protects sensitive information.

12.5 Repository Organization

A well-structured repository is easier to understand.

Example:

Project/

├── src/

├── docs/

├── tests/

├── assets/

├── README.md

├── LICENSE

└── .gitignore

Recommended folders:

- `src` → Source code
- `docs` → Documentation
- `tests` → Test files
- `assets` → Images and static resources

12.6 README File Best Practices

Every repository should contain a detailed `README.md`.

Include:

- Project Title
- Description
- Features
- Installation Steps
- Usage Instructions
- Screenshots
- Technologies Used
- License
- Author Information

Benefits:

- Helps new contributors.
- Improves project documentation.
- Makes repositories professional.

12.7 Repository Security

Never commit sensitive information.

Examples:

❌ Passwords

❌ API Keys

❌ Database Credentials

❌ Private Certificates

Instead:

Use:

.env

Add `.env` to `.gitignore`.

Example:

.env

12.8 Branch Protection

Professional teams protect the `main` branch.

Common rules:

- No direct pushes.
- Pull Request required.
- Code review required.
- Status checks must pass.
- Approved reviews required.

Benefits:

- Prevents accidental mistakes.
- Improves software quality.
- Protects production code.

12.9 Repository Maintenance

Regular maintenance tasks include:

- Delete merged branches.
- Archive inactive repositories.
- Update documentation.
- Remove unused files.
- Keep dependencies updated.
- Review open Issues.
- Close completed Pull Requests.

12.10 Professional Git Workflow

Clone Repository

Create Feature Branch

Develop

Commit

Push

Pull Request

Code Review

Merge

12.11 Git Security Best Practices

- Enable Two-Factor Authentication (2FA) on GitHub.
- Use SSH keys instead of passwords where possible.
- Keep Git updated.
- Review third-party access to repositories.
- Scan repositories for secrets before pushing.
- Use Private repositories for confidential projects.
- Limit collaborator permissions.

12.12 Common Best Practices Summary

Practice	Benefit
Meaningful commit messages	Easier history tracking
Feature branches	Safer development
`.gitignore`	Cleaner repositories
README	Better documentation
Branch protection	Prevents accidental changes
Pull Requests	Better collaboration
Code Reviews	Improved code quality
Security practices	Protect sensitive data

12.13 Common Mistakes

❌ Committing passwords or API keys.

❌ Using vague commit messages.

❌ Working directly on the `main` branch.

❌ Ignoring `.gitignore`.

❌ Creating very large Pull Requests.

❌ Leaving unused branches.

❌ Not documenting the project.

Real-Time Scenario

A development team is building a **Hospital Management System**.

Workflow:

1. Create a feature branch:

git checkout -b feature/patient-records

2. Develop the feature.
3. Commit changes:

git commit -m "Added patient records module"

4. Push the branch:

git push origin feature/patient-records

5. Create a Pull Request.
6. Team reviews the code.
7. After approval, merge into `main`.
8. Delete the merged feature branch.

This workflow ensures high-quality, well-organized development.

Interview Questions

1. Why are meaningful commit messages important?

Answer:

Meaningful commit messages clearly describe the purpose of changes, making it easier to understand project history, review code, and troubleshoot issues.

---

2. What is the purpose of `.gitignore`?

Answer:

The `.gitignore` file specifies which files and directories Git should ignore, preventing unnecessary or sensitive files from being committed.

---

3. Why should developers avoid working directly on the `main` branch?

Answer:

Working directly on the `main` branch increases the risk of introducing bugs into production. Feature branches isolate development and make testing and code reviews easier.

---

4. Why is a README file important?

Answer:

A README file provides essential information about a project, including its purpose, installation, usage, and contribution guidelines, making it easier for others to understand and use the project.

---

5. What are branch protection rules?

Answer:

Branch protection rules are GitHub settings that prevent direct changes to important branches and require actions such as Pull Requests, code reviews, and successful status checks before merging.

Practical Lab

Task 1

Create a `.gitignore` file for a Node.js project.

Task 2

Write five meaningful commit messages for different project updates.

Task 3

Create feature, bugfix, and release branches using professional naming conventions.

Task 4

Prepare a professional `README.md` for one of your GitHub projects.

Task 5

Enable branch protection rules for the `main` branch in a GitHub repository (if you have repository admin access).

Module 13: Real-World Git Workflows

Learning Objectives

After completing this module, you will be able to:

- Understand the importance of Git workflows.
- Learn the Feature Branch Workflow.
- Understand GitHub Flow.
- Learn Git Flow.
- Understand the Forking Workflow.
- Learn how to contribute to open-source projects.
- Choose the right workflow for different project types.
- Apply professional Git workflows in real-world software development.

13.1 Introduction

A **Git Workflow** is a set of rules and best practices that define how developers use Git and GitHub to collaborate efficiently.

Without a proper workflow:

- Code conflicts increase.
- Team collaboration becomes difficult.
- Releases become unstable.
- Project history becomes confusing.

Professional organizations use standardized workflows to ensure consistency and maintain software quality.

Definition

A **Git Workflow** is a structured process that defines how developers create branches, make changes, review code, and merge updates into a shared repository.

Real-Time Example

A company develops a Banking Application with 25 developers.

Different teams work on:

- Customer Management
- Payments
- Loan Services
- Security
- Mobile App

By following a Git workflow, each team develops independently while ensuring that only tested and reviewed code reaches the production branch.

13.2 Why Git Workflows are Important

Benefits include:

- Better collaboration.
- Cleaner repository history.
- Fewer merge conflicts.
- Easier project management.
- Faster code reviews.
- Stable software releases.
- Improved productivity.

13.3 Feature Branch Workflow

The **Feature Branch Workflow** is one of the simplest and most popular Git workflows.

Each new feature is developed in its own branch.

Workflow:

main

Create Feature Branch

Develop Feature

Commit Changes

Push Branch

Pull Request

Review

Merge into main

Advantages

- Isolates new features.
- Simplifies testing.
- Keeps the main branch stable.
- Easy to review.

13.4 GitHub Flow

GitHub Flow is a lightweight workflow used by many startups and web application teams.

Steps:

1. Create a branch.
2. Develop the feature.
3. Commit changes.
4. Push to GitHub.
5. Create a Pull Request.
6. Review code.
7. Merge into the main branch.
8. Deploy.

Architecture:

```
main
 │
 ├── feature/login
 │
 ├── feature/payment
 │
 └── feature/profile
```

Best For

- Web applications.
- Continuous Deployment.
- Agile development.

13.5 Git Flow

Git Flow is a structured workflow for large software projects.

It uses multiple long-lived branches.

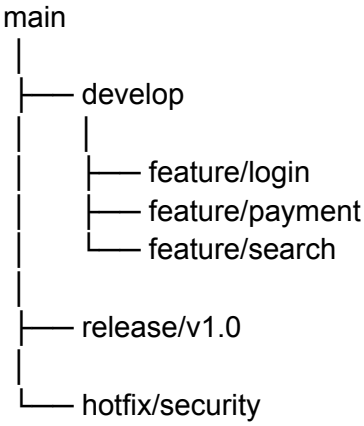
Main Branches:

- `main`
- `develop`

Supporting Branches:

- `feature/*`
- `release/*`
- `hotfix/*`

Architecture:



Advantages

- Well-organized releases.
- Supports multiple teams.
- Ideal for enterprise projects.

Disadvantages

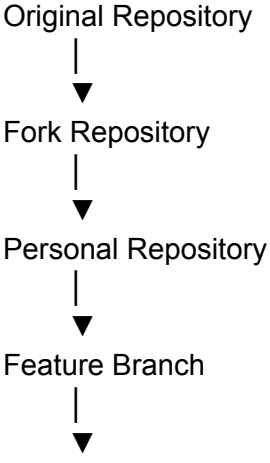
- More complex.
- Requires discipline.
- Overhead for small projects.

13.6 Forking Workflow

The **Forking Workflow** is commonly used in open-source projects.

Instead of working directly in the original repository, contributors create a personal copy (fork).

Workflow:



Pull Request

Advantages

- Safe collaboration.
- No direct write access required.
- Encourages open-source contributions.

13.7 Open Source Contribution Workflow

Typical process:

1. Find a repository.
2. Fork the repository.
3. Clone your fork.
4. Create a feature branch.
5. Make changes.
6. Commit changes.
7. Push to your fork.
8. Create a Pull Request.
9. Address review comments.
10. Merge after approval.

Example Commands:

```
git clone https://github.com/yourusername/project.git
git checkout -b feature/improve-readme
git push origin feature/improve-readme
```

13.8 Choosing the Right Workflow

Workflow	Best For
Feature Branch Workflow	Small to medium teams
GitHub Flow	Web applications and Agile teams
Git Flow	Enterprise applications with planned releases
Forking Workflow	Open-source projects

13.9 Workflow Comparison

Feature	Feature Branch	GitHub Flow	Git Flow	Forking
Easy to Learn	✅	✅	❌	✅
Supports Releases	⚠️	⚠️	✅	⚠️
Enterprise Ready	✅	✅	✅	✅
Open Source	⚠️	⚠️	⚠️	✅
Continuous Deployment	⚠️	✅	⚠️	⚠️

13.10 Professional Development Lifecycle

Requirement
↓
Create Branch
↓
Develop Feature
↓
Commit
↓
Push
↓
Pull Request
↓
Review
↓
Merge
↓
Deploy

13.11 Best Practices

- Choose a workflow suitable for your team.
- Keep feature branches focused on a single task.
- Pull the latest changes frequently.
- Review every Pull Request.
- Delete merged branches.
- Write clear commit messages.
- Test thoroughly before merging.
- Keep the `main` branch stable.

13.12 Common Mistakes

❌ Working directly on the `main` branch.

❌ Creating very large feature branches.

❌ Ignoring Pull Request reviews.

❌ Keeping stale branches for long periods.

❌ Merging untested code into production.

Real-Time Scenario

A software company develops an **Online Learning Platform**.

Workflow:

1. A developer creates a branch:

git checkout -b feature/course-search

2. The feature is developed.
3. Changes are committed:

git commit -m "Added course search functionality"

4. The branch is pushed:

git push origin feature/course-search

5. A Pull Request is created.
6. The team reviews the code.
7. After approval, the branch is merged into `main`.
8. The application is deployed.

This workflow is commonly followed by Agile software teams.

Interview Questions

1. What is a Git Workflow?

Answer:

A Git Workflow is a structured process that defines how developers use Git to create branches, manage changes, review code, and merge updates into a shared repository.

---

2. What is the Feature Branch Workflow?

Answer:

The Feature Branch Workflow requires each new feature to be developed in its own branch, allowing isolated development and easier code reviews.

---

3. What is GitHub Flow?

Answer:

GitHub Flow is a lightweight workflow where developers create a branch, make changes, open a Pull Request, review the code, merge it into `main`, and deploy.

---

4. When should Git Flow be used?

Answer:

Git Flow is ideal for enterprise applications with multiple teams, planned releases, and long-term maintenance because it provides structured branching for features, releases, and hotfixes.

---

5. What is the Forking Workflow?

Answer:

The Forking Workflow is commonly used in open-source development, where contributors create their own copy (fork) of a repository, make changes independently, and submit them through Pull Requests.

Practical Lab

Task 1

Create a feature branch using the Feature Branch Workflow.

Task 2

Simulate a GitHub Flow by creating a Pull Request from a feature branch.

Task 3

Design a Git Flow branch structure for a banking application.

Task 4

Fork an open-source repository on GitHub and clone it locally.

Task 5

Make a documentation improvement in your fork and create a Pull Request.

Module 14: Git & GitHub Projects

Learning Objectives

After completing this module, you will be able to:

- Build professional GitHub repositories.
- Upload different types of projects to GitHub.
- Organize repositories professionally.
- Collaborate on team projects.
- Contribute to open-source projects.
- Create an impressive GitHub portfolio.
- Follow industry-standard project workflows.
- Showcase projects for internships and placements.

14.1 Introduction

Employers don't just look at resumes—they also evaluate your **GitHub profile**.

A well-maintained GitHub profile demonstrates:

- Programming skills
- Project experience
- Collaboration ability
- Version control knowledge
- Consistent learning

A strong GitHub profile often increases your chances of internships and software development jobs.

Definition

A **GitHub Project** is a software application or code repository hosted on GitHub that demonstrates programming skills, collaboration, documentation, and version control practices.

Real-Time Example

A student applies for a Software Developer internship.

The recruiter reviews:

- Resume
- GitHub Profile
- Personal Projects
- Commit History

- README files
- Contribution Activity

The student with a well-organized GitHub portfolio has a higher chance of being shortlisted.

14.2 Types of GitHub Projects

Developers can showcase various types of projects.

Examples:

- Web Applications
- Mobile Applications
- Python Projects
- Machine Learning Projects
- Data Science Projects
- API Projects
- DevOps Projects
- Cloud Projects
- Open Source Contributions

14.3 Creating a Professional Repository

A professional repository should include:

```
Project/
│
├── src/
├── docs/
├── tests/
├── assets/
├── README.md
├── LICENSE
├── .gitignore
└── CONTRIBUTING.md
```

Purpose of Files

- **README.md** → Project overview
- **LICENSE** → Usage permissions
- **.gitignore** → Ignore unnecessary files
- **CONTRIBUTING.md** → Contribution guidelines

14.4 Writing a Professional README

A README is the first thing visitors see.

Recommended Sections:

Project Title

Project Description

Features

Installation

Usage

Screenshots

Technologies Used

Folder Structure

Future Enhancements

License

Author

Example

Student Management System

A web application for managing student records.

Features

- Login
- Dashboard
- Attendance
- Reports

14.5 Uploading a Local Project to GitHub

Step 1

Initialize Git.

git init

Step 2

Add files.

git add .

Step 3

Commit changes.

git commit -m "Initial project upload"

Step 4

Connect the remote repository.

git remote add origin https://github.com/username/project.git

Step 5

Push the project.

git push -u origin main

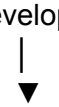
14.6 Team Collaboration Project

Professional team workflow:

Create Repository

Invite Team Members

Create Feature Branch

Develop Feature


Commit

Push

Pull Request

Review

Merge

14.7 Open Source Contribution

Contributing to open-source projects helps improve coding skills and visibility.

Workflow:

1. Find a repository.
2. Fork the repository.
3. Clone your fork.
4. Create a branch.
5. Make changes.
6. Commit changes.
7. Push your branch.
8. Create a Pull Request.

14.8 Portfolio Projects

Recommended beginner-to-intermediate projects:

Web Development

- Personal Portfolio
- To-Do App
- Weather App
- Expense Tracker
- Blog Website

Python

- Library Management System
- Student Management System
- Calculator
- File Organizer
- Password Generator

Data Science

- Sales Dashboard
- Customer Churn Prediction
- House Price Prediction
- Movie Recommendation System
- COVID Data Analysis

DevOps

- Dockerized Web Application
- Jenkins CI/CD Pipeline
- Kubernetes Deployment
- Linux Automation Scripts
- Monitoring with Prometheus and Grafana

14.9 GitHub Portfolio Best Practices

A strong GitHub profile should have:

- Professional profile picture.
- Clear bio.
- Pinned repositories.
- Detailed README files.
- Frequent commits.
- Consistent activity.
- Meaningful repository names.
- Proper documentation.

14.10 Repository Checklist

Before making a repository public, verify:

✅ README exists.

✅ License added.

✅ `.gitignore` configured.

✅ No sensitive information.

✅ Meaningful commit history.

✅ Proper folder structure.

✅ Screenshots included.

✅ Installation guide available.

14.11 Professional Project Workflow

Requirement

Planning

Create Repository

Development

Commit

Push

Pull Request

Review


▼
Merge
|
▼
Release

14.12 GitHub Portfolio Tips

- Pin your best repositories.
- Keep repository names simple.
- Add project screenshots.
- Write clear documentation.
- Update projects regularly.
- Showcase different technologies.
- Contribute to open-source projects.
- Keep commit history clean.

14.13 Common Mistakes

❌ Uploading incomplete projects.

❌ Missing README files.

❌ Committing passwords or API keys.

❌ Poor repository organization.

❌ Large, unclear commit history.

❌ No documentation.

❌ Ignoring Pull Requests.

Real-Time Scenario

A final-year Computer Science student builds a **Student Attendance Management System**.

Steps:

1. Create a GitHub repository.
2. Upload the project using Git.
3. Add a detailed README.
4. Include screenshots.
5. Add a LICENSE.
6. Configure `.gitignore`.
7. Pin the repository to the GitHub profile.
8. Share the repository link in the resume and LinkedIn profile.

During placement interviews, recruiters review the project and appreciate the professional documentation and commit history.

Interview Questions

1. Why is a GitHub portfolio important?

Answer:

A GitHub portfolio showcases your coding skills, project experience, collaboration ability, and familiarity with Git. It helps recruiters evaluate your practical knowledge beyond your resume.

---

2. What should every professional repository contain?

Answer:

A professional repository should include:

- `README.md`
- `LICENSE`
- `.gitignore`
- Source code
- Documentation
- Tests (if applicable)
- Clear folder structure

---

3. Why is a README important?

Answer:

A README provides an overview of the project, installation steps, usage instructions, technologies used, and contribution guidelines, making the repository easier to understand and use.

---

4. Why should developers contribute to open-source projects?

Answer:

Open-source contributions improve coding skills, provide real-world collaboration experience, help build a professional reputation, and strengthen a developer's portfolio.

---

5. What makes a GitHub repository attractive to recruiters?

Answer:

Recruiters look for clean code, meaningful commit history, clear documentation, active maintenance, proper project organization, and evidence of practical problem-solving.

Practical Lab

Task 1

Create a new GitHub repository named **Portfolio-Projects**.

Task 2

Upload one of your existing projects with a complete `README.md`.

Task 3

Add a `.gitignore` file and a suitable LICENSE.

Task 4

Add screenshots and usage instructions to the repository.

Task 5

Fork an open-source project, make a small documentation improvement, and submit a Pull Request.

Module 15: Git & GitHub Interview Preparation & Career Guidance

Learning Objectives

After completing this module, you will be able to:

- Revise all Git and GitHub concepts.
- Prepare for technical interviews.
- Solve real-world Git scenarios.
- Learn troubleshooting techniques.
- Understand Git usage in software companies.
- Build confidence for internships and placements.
- Plan a career path using Git and GitHub skills.

15.1 Introduction

Git and GitHub are among the most essential skills for software developers, DevOps engineers, cloud engineers, and data scientists.

Almost every software company uses Git for:

- Version Control
- Team Collaboration
- Continuous Integration (CI)
- Continuous Deployment (CD)
- Code Reviews
- Release Management

Mastering Git and GitHub not only improves productivity but also increases employability.

Definition

Git Interview Preparation is the process of revising Git concepts, practicing commands, understanding workflows, and solving practical scenarios to perform confidently in technical interviews and real-world development.

Real-Time Example

A final-year student attends a software developer interview.

The interviewer asks:

- Explain Git Branching.
- Differentiate between Git Fetch and Git Pull.
- Resolve a merge conflict.
- Describe your GitHub projects.
- Explain a Pull Request workflow.

A candidate with practical Git experience confidently answers these questions and demonstrates projects hosted on GitHub.

15.2 Git Revision Roadmap

Version Control

Git Basics

Repositories

Git Commands

Branching

GitHub

Collaboration

Advanced Git

15.3 Essential Git Commands Revision

Category	Important Commands
Repository	`git init`, `git clone`, `git status`
Staging	`git add`, `git restore`
Commit	`git commit`, `git log`, `git diff`
Branching	`git branch`, `git switch`, `git merge`
Remote	`git remote`, `git push`, `git pull`, `git fetch`
Advanced	`git stash`, `git reset`, `git revert`, `git rebase`, `git cherry-pick`, `git tag`

15.4 Most Frequently Asked Git Interview Questions

1. What is Git?

Answer:

Git is a distributed version control system that tracks changes in source code and enables collaborative software development.

---

2. What is GitHub?

Answer:

GitHub is a cloud-based platform that hosts Git repositories and provides collaboration tools such as Pull Requests, Issues, Projects, and Actions.

---

3. What is the difference between Git and GitHub?

Git	GitHub
Version Control System	Repository Hosting Platform
Works locally	Cloud-based
Tracks code history	Enables collaboration

---

4. What is a Branch?

Answer:

A branch is an independent line of development that allows developers to work on new features or bug fixes without affecting the main branch.

---

5. What is a Pull Request?

Answer:

A Pull Request is a request to merge changes from one branch into another after review and approval.

---

6. What is the difference between `git fetch` and `git pull`?

Answer:

- `git fetch` downloads changes without merging them.
- `git pull` downloads and automatically merges changes into the current branch.

---

7. What is `git stash`?

Answer:

`git stash` temporarily saves uncommitted changes so that developers can switch branches or perform other tasks without losing their work.

---

8. What is the difference between `git reset` and `git revert`?

Answer:

- `git reset` rewrites commit history and is typically used for local changes.
- `git revert` creates a new commit that reverses a previous commit while preserving project history.

---

9. What is Git Rebase?

Answer:

Git Rebase moves or reapplies commits onto another branch to create a cleaner, linear commit history.

---

10. What is `.gitignore`?

Answer:

`.gitignore` specifies files and directories that Git should ignore and not track.

15.5 Real-World Git Scenarios

Scenario 1: Merge Conflict

Situation:

Two developers modify the same line of a file.

Solution:

- Pull the latest changes.
- Resolve the conflict manually.
- Stage the resolved file.
- Commit the merge.

Scenario 2: Wrong Commit Message

Situation:

A commit message contains a typo.

Solution:

git commit --amend

Scenario 3: Accidentally Deleted a Branch

Situation:

A feature branch is deleted.

Solution:

Recover it using the commit history (for example, by locating the commit in the log or reflog, if available) and recreate the branch.

Scenario 4: Need to Switch Tasks

Situation:

You have unfinished work but must fix an urgent bug.

Solution:

git stash

Restore later:

git stash pop

Scenario 5: Wrong File Committed

Situation:

A confidential file was committed accidentally.

Solution:

Remove the sensitive file from the repository history before sharing it and rotate any exposed credentials. Also update `.gitignore` to prevent it from being committed again.

15.6 Git Troubleshooting Guide

Problem	Solution
Merge Conflict	Resolve conflicts manually
Wrong Branch	Switch to the correct branch
Push Rejected	Pull latest changes, resolve conflicts if needed, then push again
Detached HEAD	Checkout or switch back to a branch
Forgotten Commit	Use `git log` to locate it

15.7 Git Career Opportunities

Git is required in many technical roles.

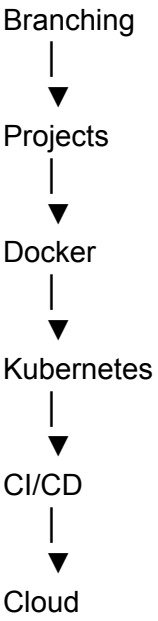
Popular Careers:

- Software Developer
- Full Stack Developer
- Backend Developer
- Frontend Developer
- DevOps Engineer
- Cloud Engineer
- Site Reliability Engineer (SRE)
- Data Scientist
- Machine Learning Engineer
- Cybersecurity Engineer

15.8 Git Learning Roadmap

Git Basics

GitHub



15.9 Git Cheat Sheet

Task	Command
Initialize Repository	`git init`
Clone Repository	`git clone`
Check Status	`git status`
Stage Files	`git add .`
Commit Changes	`git commit -m "message"`
View History	`git log --oneline`
Create Branch	`git branch feature-name`
Switch Branch	`git switch feature-name`
Merge Branch	`git merge feature-name`
Push Changes	`git push`

Pull Changes	`git pull`
Fetch Updates	`git fetch`
Save Work	`git stash`
Restore Work	`git stash pop`

15.10 Best Practices Before an Interview

- Practice Git commands daily.
- Create multiple GitHub projects.
- Write professional README files.
- Learn branching and merging thoroughly.
- Understand Pull Requests.
- Practice resolving merge conflicts.
- Keep your GitHub profile active.
- Contribute to at least one open-source project.

15.11 Common Interview Mistakes

❌ Memorizing commands without understanding them.

❌ Not practicing GitHub workflows.

❌ Having an incomplete GitHub profile.

❌ Using vague commit messages.

❌ Not knowing the difference between `git fetch` and `git pull`.

❌ Being unable to explain personal projects.

Real-Time Scenario

A recruiter asks a candidate to demonstrate Git knowledge.

The candidate:

1. Creates a repository.
2. Creates a feature branch.
3. Commits changes.
4. Pushes the branch.
5. Opens a Pull Request.
6. Explains the workflow.
7. Shows GitHub projects.

The candidate's practical demonstration leaves a strong impression.

Interview Questions

1. Why is Git preferred over traditional version control systems?

Answer:

Git is distributed, fast, supports offline work, provides powerful branching and merging capabilities, and enables efficient collaboration.

---

2. What happens when you run `git commit`?

Answer:

Git creates a new commit object containing a snapshot of the staged changes along with metadata such as the author, timestamp, and commit message.

---

3. What is a merge conflict?

Answer:

A merge conflict occurs when Git cannot automatically combine changes because multiple branches modified the same part of a file.

---

4. What is the purpose of GitHub Actions?

Answer:

GitHub Actions automates workflows such as building, testing, and deploying applications.

---

5. How would you describe your Git workflow during an interview?

Answer:

A typical workflow is:

- Clone the repository.
- Create a feature branch.
- Develop the feature.
- Commit changes with meaningful messages.
- Push the branch.
- Create a Pull Request.
- Review and merge after approval.

Practical Lab

Task 1

Create a new GitHub repository and upload a sample project.

Task 2

Create a feature branch, commit changes, and merge it into the `main` branch.

Task 3

Simulate a merge conflict and resolve it.

Task 4

Create a release tag named `v1.0.0`.

Task 5

Prepare your GitHub profile by adding:

- Professional profile photo.

- Bio.
- Pinned repositories.
- README files.
- Portfolio projects.